

Practical No.:	04
Practical Title :	To Build the pipeline of jobs using Maven / Gradle / Ant in Jenkins, create a pipeline script to Test and deploy an application over the tomcat server.
Aim:	To create and implement a CI/CD pipeline in Jenkins using Maven/Gradle/Ant and deploy an application on Apache Tomcat.
Objective:	To understand Continuous Integration and Continuous Deployment (CI/CD). To create a pipeline job in Jenkins. To automate the build process using Maven. To perform automatic testing of an application

Theory:**Theory****What is Jenkins Pipeline?**

Jenkins Pipeline is a combination of Plugins which automates number of tasks and makes the CI/CD pipeline efficient, high in quality and reliable.

What is Jenkinsfile?

Jenkins file is nothing but a simple text file which is used to write the Jenkins Pipeline and to automate the Continuous Integration process.

Jenkinsfile works as a “Pipeline as a Code”.

Jenkinsfile is written in couple of way

- Declarative pipeline syntax
- Scripted pipeline syntax

1: Login to Jenkins

First thing, we will login to our Jenkins account

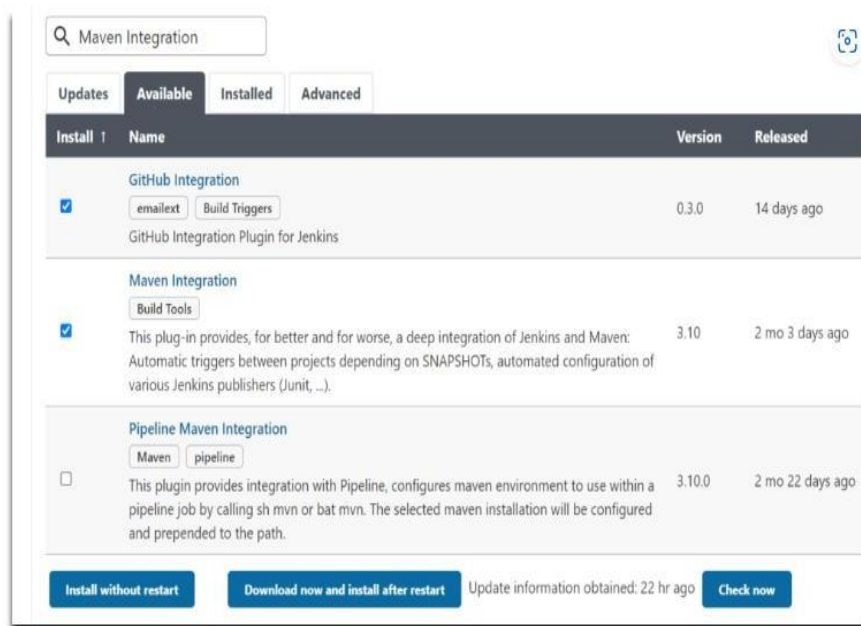
**2. Install GitHub Integration and Maven Plugins in Jenkins**

To build Java project with maven we need to install some plugins like,

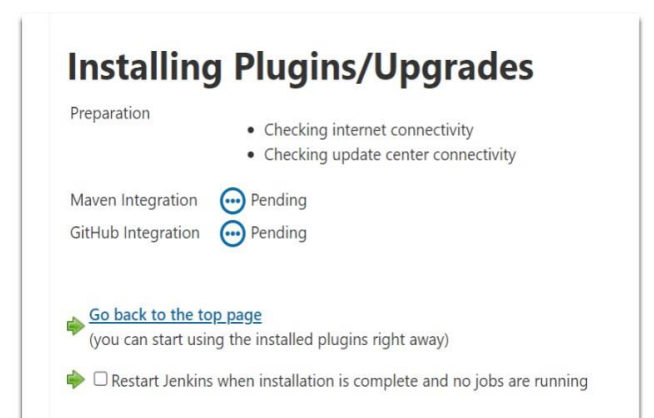
- Git Plugin
- Pipeline Plugin
- Maven Integration Plugin

Navigate to Dashboard->> Manage Jenkins ->> Plugins ->> Available Plugins

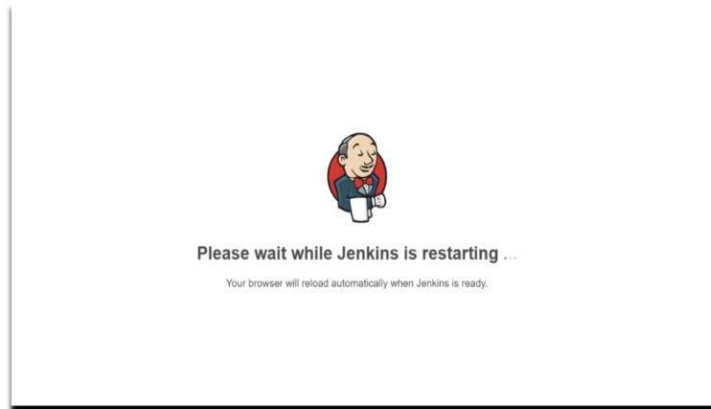
Now search for “GitHub Integration Plugin” and click on Download now and install after restart.



Once the plugins are downloaded successfully. So now click on Restart Jenkins checkbox.



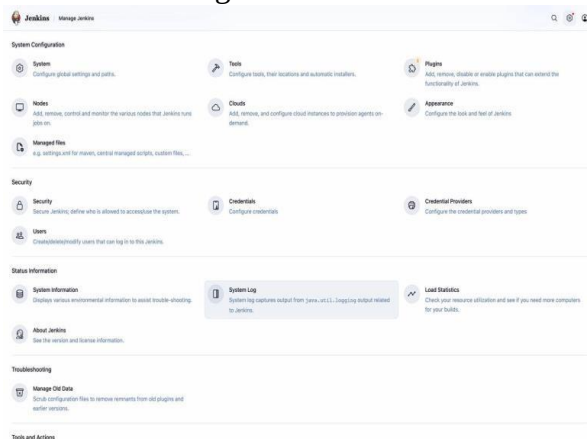
Please wait while Jenkins is restarting



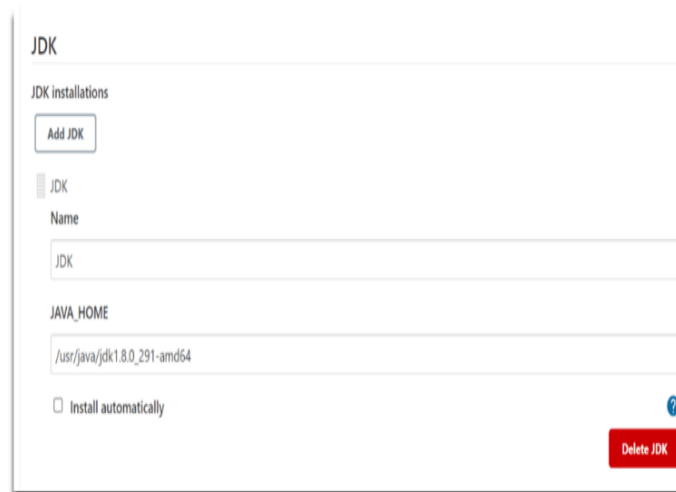
3. Global Tool configuration in Jenkins

Now we will configure System by adding JDK and Maven installation in Jenkins.

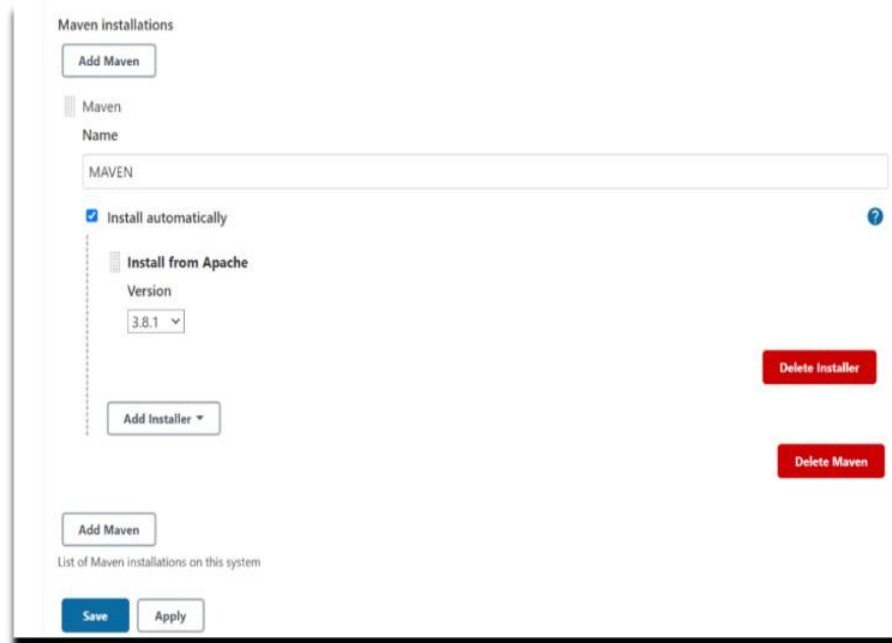
Navigate to Dashboard->> Manage Jenkins->> Tools



Click on JDK->> Add JDK, Give a JDK name and provide your JAVA_HOME path where the JDK 21 is present.

A screenshot of the "JDK" configuration page in Jenkins. It shows the "JDK installations" section with an "Add JDK" button. Below this, there is a form for adding a new JDK. The form has a "Name" field with the value "JDK" and a "JAVA_HOME" field with the value "/usr/java/jdk1.8.0_291-amd64". There is an unchecked checkbox for "Install automatically" and a red "Delete JDK" button at the bottom right.

Now below click on **Maven->> Add Maven**, We can provide our Installed Maven path or we can also use Jenkins default one. In this case we are using Jenkins default Maven version.

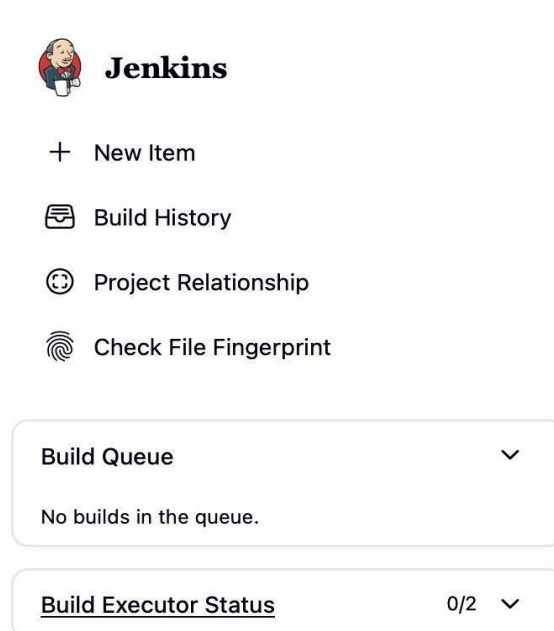


The screenshot shows the 'Maven installations' configuration page in Jenkins. At the top, there is an 'Add Maven' button. Below it, a 'Maven' section is expanded, showing a 'Name' field with the value 'MAVEN'. The 'Install automatically' checkbox is checked. Under 'Install from Apache', the 'Version' dropdown is set to '3.8.1'. There are 'Delete Installer' and 'Delete Maven' buttons on the right. At the bottom, there is an 'Add Maven' button, a 'List of Maven installations on this system' section, and 'Save' and 'Apply' buttons.

Click on apply and save.

4. Create Pipeline Job in Jenkins

Now we will create a new Pipeline Job in Jenkins, So, click on “New Item” on the Jenkins Dashboard as shown below.



Enter Job name and select “Pipeline”, Click OK.

New Item

Enter an item name

DemoPipelineTask

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- External Job**
This type of job allows you to record the execution of a process run outside Jenkins, even on a remote machine. This is designed so that you can use Jenkins as a dashboard of your existing automation system.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

5. Enter Project details in Jenkins Pipeline

Provide some description for the newly created job in General Section, Click on GitHub project and provide the GitHub URL.

You can use Sample Java Project on GitHub with Jenkinsfile

Jenkins / DemoPipelineTask / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

General

Enabled

Description

Plain text Preview

☐ Discard old builds ?

☐ Do not allow concurrent builds

☐ Do not allow the pipeline to resume if the controller restarts

☒ GitHub project

Project url ?

https://github.com/sarvesh871/MavenProject.git

Advanced

☐ Pipeline speed/durability override ?

☐ Preserve stashes from completed builds ?

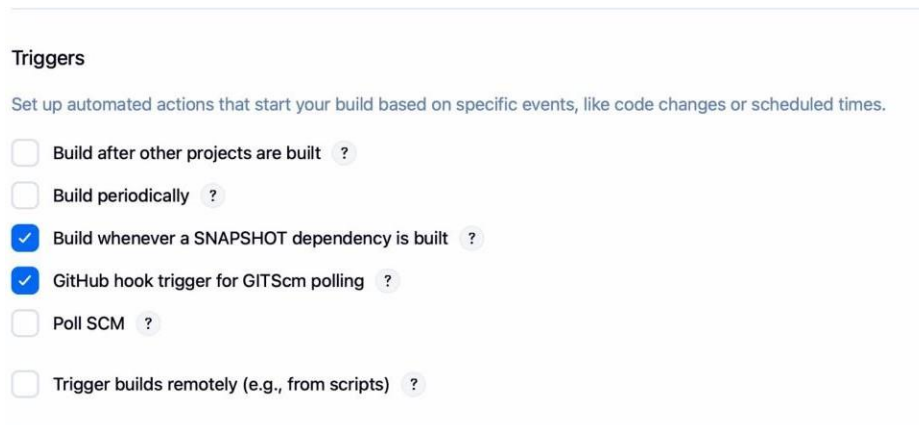
☐ This project is parameterized ?

☐ Throttle builds ?

Save Apply

Now we will set the Build Triggers, Select:

- Build whenever a SNAPSHOT dependency is built.
- GitHub hook trigger for GITScm polling.



The screenshot shows the 'Triggers' section of the Jenkins configuration page. It includes a sub-header 'Triggers' and a description: 'Set up automated actions that start your build based on specific events, like code changes or scheduled times.' Below this are several checkboxes with corresponding labels and help icons:

- ☐ Build after other projects are built ?
- ☐ Build periodically ?
- ☒ Build whenever a SNAPSHOT dependency is built ?
- ☒ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?
- ☐ Trigger builds remotely (e.g., from scripts) ?

Now in Pipeline section click on drop down and select Pipeline script from SCM.



The screenshot shows the 'Pipeline' section of the Jenkins configuration page. It includes a sub-header 'Pipeline' and a description: 'Define your Pipeline using Groovy directly or pull it from source control.' Below this is a 'Definition' section with a dropdown menu. The dropdown is open, showing 'Pipeline script from SCM' as the selected option.

In SCM select Git and below it provide your Git repository URL where your project is located and add your GitHub credentials.



The screenshot shows the 'SCM' section of the Jenkins configuration page. It includes a sub-header 'SCM' and a description: 'Define your Pipeline using Groovy directly or pull it from source control.' Below this is a 'Definition' section with a dropdown menu. The dropdown is open, showing 'Pipeline script from SCM' as the selected option. Below the dropdown is a 'Repositories' section with a dropdown menu. The dropdown is open, showing 'Git' as the selected option. Below the dropdown is a 'Repository URL' field with the value 'https://github.com/sarvesh871/MavenProject.git'. Below the URL field is a 'Credentials' section with a dropdown menu. The dropdown is open, showing 'GitHub credentials for Jenkins pipeline' as the selected option. Below the credentials dropdown is an 'Advanced' section with a dropdown menu.

By default Jenkins provide branch as Master but you can change it to dev, main, or to whatever your branch is.(if you don't know you can check the branch by going to your GitHub repository OR by running git branch command on your shell).
Our branch is main branch so will change master to main

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Now comes the most important part of your Project / Job configuration, Provide accurate path of your Jenkinsfile.

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

Click on Apply and Save.

6. Jenkinsfile to Build Java Project using Maven in Jenkins Pipeline

Now we see how to write a Declarative Pipeline script to build java project using maven.

Why Declarative Pipeline?

Declarative Pipeline is a latest method mostly used in now a day to write Pipeline rather than that typical way (Scripted) because Declarative provides new feature which can even be tested in Git for source control)

Below is Jenkinsfile to Build Java Project using Maven in Jenkins and generate junit test report.

Jenkinsfile code:

```
pipeline {
    agent any

    tools {
        maven 'Maven' // Must match Jenkins Global Tool Configuration
        jdk 'JDK'      // Configure this as JDK 21 in Jenkins
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
    }
}
```

```
stage('Build') {
    steps {
        script {
            if (isUnix()) {
                sh 'mvn clean compile'
            } else {
                bat 'mvn clean compile'
            }
        }
    }
}

stage('Test') {
    steps {
        script {
            if (isUnix()) {
                sh 'mvn test'
            } else {
                bat 'mvn test'
            }
        }
    }
}

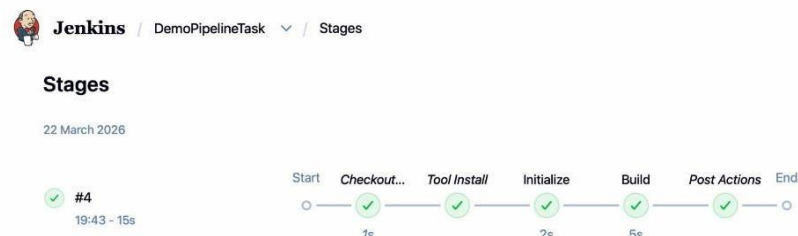
post {
    always {
        junit '**/target/surefire-reports/*.xml'
    }
}
}
```

Note- Replace and add your workspace path in dir.

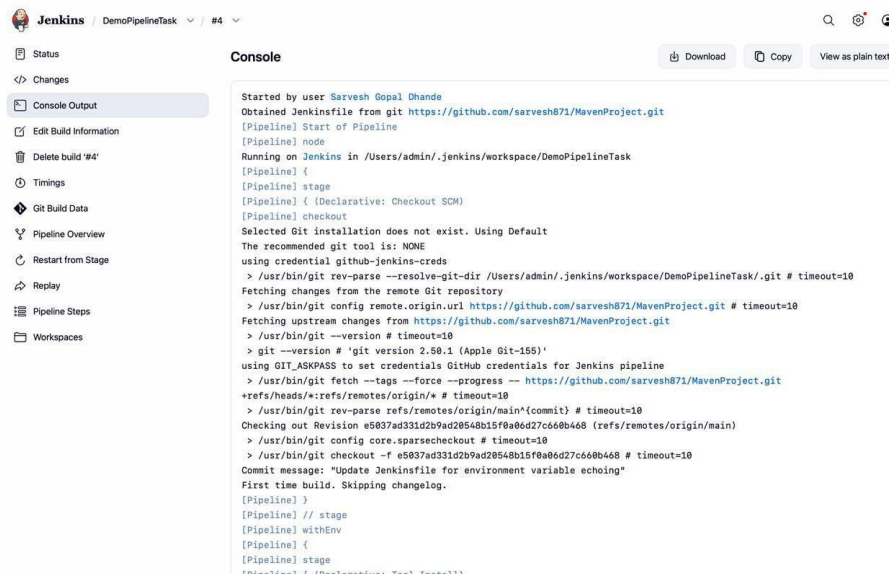
Build Java Project using Maven in Jenkins Pipeline

Now open your Jenkins Dashboard and click on your job, now click on Build Now

You can see Jenkins Pipeline Build stages.



Check the console output to check Jenkins pipeline logs.



The screenshot shows the Jenkins web interface for a pipeline named 'DemoPipelineTask'. The left sidebar contains navigation links: Status, Changes, Console Output (selected), Edit Build Information, Delete build #4, Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, and Workspaces. The main area displays the 'Console' output, which includes the following text:

```
Started by user Sarvesh Gopal Dhande
Obtained Jenkinsfile from git https://github.com/sarvesh871/MavenProject.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /Users/admin/.jenkins/workspace/DemoPipelineTask
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
using credential github-jenkins-creds
> /usr/bin/git rev-parse --resolve-git-dir /Users/admin/.jenkins/workspace/DemoPipelineTask/.git # timeout=10
Fetching changes from the remote Git repository
> /usr/bin/git config remote.origin.url https://github.com/sarvesh871/MavenProject.git # timeout=10
Fetching upstream changes from https://github.com/sarvesh871/MavenProject.git
> /usr/bin/git --version # timeout=10
> git --version # 'git version 2.50.1 (Apple Git-155)'
using GIT_ASKPASS to set credentials GitHub credentials for Jenkins pipeline
> /usr/bin/git fetch --tags --force --progress -- https://github.com/sarvesh871/MavenProject.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> /usr/bin/git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision e5037ad331d2b9ad20548b15f0a86d27c660b468 (refs/remotes/origin/main)
> /usr/bin/git config core.sparsecheckout # timeout=10
> /usr/bin/git checkout -f e5037ad331d2b9ad20548b15f0a86d27c660b468 # timeout=10
Commit message: "Update Jenkinsfile for environment variable echoing"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] # /usr/bin/git rev-parse --resolve-git-dir /Users/admin/.jenkins/workspace/DemoPipelineTask/.git # timeout=10
```

We have covered How to Build Java Project using Maven in Jenkins Pipeline.

In this article we studied How to Build Java Project Using Maven in Jenkins Pipeline. First we learned how to add a Build Stage and print message. Next we done with adding the tools like JDK and Maven. Then at last we explored how to Run a Maven Build with JUnit.

Conclusion:
